

Solution to Homework 4

Meshach Aruoriwo Ogbor,
Md Rasel

November 7, 2023

1 Exercise 4.1 - Lattice Walk

```
1 #include <mpi.h>
2 #include <stdio.h>
3
4 #define MAX_DIRECTIONS 1000
5
6 int main(int argc, char* argv[]) {
7
8     int rank, numprocs;
9
10    MPI_Init(&argc, &argv);
11
12    MPI_Comm_rank(MPLCOMM_WORLD, &rank);
13    MPI_Comm_size(MPLCOMM_WORLD, &numprocs);
14
15    int x0 = rank % 8; // calculate x0
16    int x1 = rank / 8; // calculate x1
17
18    int neighbors[4]; // declare an array for
19                      // neighbours
20    neighbors[0] = (x0 + 1) % 8 + x1 * 8; // positive
21                      // x0
22    neighbors[1] = x0 + ((x1 + 1) % 8) * 8; //
23                      // positive x1
24    neighbors[2] = (x0 + 7) % 8 + x1 * 8; // negative
25                      // x0
```

```

22     neighbors[3] = x0 + ((x1 + 7) % 8) * 8; //
        negative x1
23
24     int pos = 0;
25     int value = 0;
26     int direction;
27
28     int directions[MAX_DIRECTIONS]; // declare an
        array for directions
29     int num_directions = 0;
30
31     // if rank = 0, open the file , otherwise print
        error and abort
32     if (rank == 0) {
33         FILE* file = fopen("directions.dat", "r");
34         if (file == NULL) {
35             perror("Error opening file");
36             MPI_Abort(MPLCOMM_WORLD, -1);
37         }
38
39         while (fscanf(file, "%d", &directions[
            num_directions]) != EOF && num_directions <
            MAX_DIRECTIONS) {
40             num_directions++;
41         }
42
43         fclose(file);
44
45         if (ferror(file)) {
46             perror("Error reading file");
47             MPI_Abort(MPLCOMM_WORLD, -1);
48         }
49     }
50
51     // broadcast the number of directions and
        directions to all processors
52     MPI_Bcast(&num_directions, 1, MPI_INT, 0,
        MPLCOMM_WORLD);
53     MPI_Bcast(directions, num_directions, MPI_INT, 0,
        MPLCOMM_WORLD);

```

```

54
55     for (int i = 0; i < num_directions; i++) {
56         direction = directions[i];
57         int n = neighbors[direction];
58
59         MPI_Bcast(&n, 1, MPI_INT, pos, MPLCOMM_WORLD)
60             ;
61
62         if (rank == pos) {
63             value += rank;
64             MPI_Send(&value, 1, MPI_INT, n, 0,
65                 MPLCOMM_WORLD);
66         }
67
68         if (rank == n) {
69             MPI_Recv(&value, 1, MPI_INT, pos, 0,
70                 MPLCOMM_WORLD, MPI_STATUS_IGNORE);
71         }
72
73         pos = n; // update the position
74     }
75
76     if (rank == 0) {
77         printf("Final value: %d\n", value); // print
78             the final value
79     }
80
81     MPI_Finalize();
82     return 0;
83 }

```

For compilation on Stromboli, we used the following command:
`mpicc -o ex4_1 ex4_1.c`

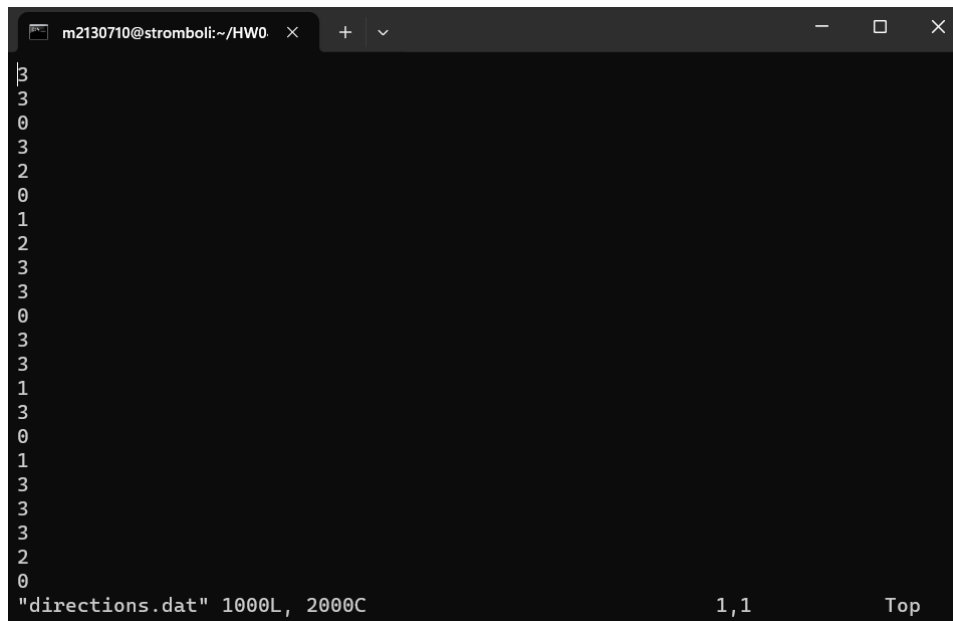
2 Submit Script for exercise 4.1

```
1 #!/bin/bash
2 #SBATCH --nodes=3
3 #SBATCH --ntasks=64
4 #SBATCH --partition=compute2011
5 #SBATCH --exclusive
6 ##SBATCH --output=output.txt
7
8 module load mpi/openmpi/4.1.0
9 mpirun ./ex4_1
```

3 Results

Final value: 25929

We have faced some difficulties while reading the `directions.dat` file. At first, we tried without `#define MAX_DIRECTIONS 1000` and we got the final value but the program continued to run for an infinite time. Due to that we defined this and it successfully ended the program. If there is any better way, we would be open for your suggestions/recommendations.



```
m2130710@stromboli:~/HW0
3
3
0
3
2
0
1
2
3
3
0
3
3
1
3
0
1
3
3
3
3
2
0
"directions.dat" 1000L, 2000C 1,1 Top
```